

Ultra short signatures with Dragon $HFE_{LL'}$

Jacques Patarin¹³, Jan Vacek²³

¹Thales DIS France SAS, Meudon, France,

²Thales DIS Czech Republic, s.r.o., Prague, Czech Republic,

³Université Paris-Saclay, UVSQ, CNRS, Laboratoire de mathématiques de Versailles,
78000, Versailles, France,

{jacques.patarin,jan.vacek}@thalesgroup.com

Abstract. We will present in this paper two new (post-Quantum) multivariate signature schemes, one with short signature, called $HFE_{LL'}$, and one with even shorter (“ultra-short”) signatures, called $D - HFE_{LL'}$. These two schemes are variants of the HFE scheme (one of the most studied multivariate schemes) with a new and specific use of the LL' perturbation. In the last years, HFE suffered new attacks, specially Min-Rank attacks, but thanks to our specific LL' perturbation, our schemes resist all know attacks.

Keywords: Post-Quantum cryptography, Multivariate Cryptography, HFE, Ultra-short signatures

1 Introduction

Multivariate Cryptography is one of the family of schemes studied at present in Post-Quantum Cryptography. Moreover, Multivariate Cryptography is known to have generally short signatures. For example, with an expected security of 128 bits, some variants of the UOV scheme submitted to the NIST Post-Quantum competition have 768 bits (UOV, [1]), 1448 bits (Mayo, [2]) or 992 bits (SNOVA, [3]).

With variants of HFE (instead of variants of UOV) even shorter signatures were suggested, for example 258 bits with the GeMMs scheme [4].

However, in the last years, HFE suffered new attacks, specially improved Min-Rank attacks ([5,6]). Due to these attacks, the GeMMs scheme is no longer a NIST candidate (unlike the scheme based on UOV).

Recently, in [7], a systematic analysis of many permutations that may be used (such as $-, IP, v$, etc. but not LL') to enforce the security of multivariate schemes has been done. Thanks to this analysis, some variants of HFE in signature that are still secure at present have been suggested, such HFE_{IP-} or HFE_{IPv} . However, with these variants the length of the signatures is (still small but) larger than before, and these schemes require more computations.

A completely different way to enforce the security of HFE in signature has been suggested in 2006 in [8]. In this paper the perturbation LL' was introduced (one LL' on each equation of the public key), and the scheme was a “probabilistic multivariate signature scheme”, i.e. not all the equations of the public key must

be satisfied, but only enough of them. Then the scheme resists MinRank attacks, but the signatures become much bigger.

Recently, in [9] it was suggested to use this LL' perturbation for encryption, instead of signatures.

In this paper we will again introduce LL' with HFE in signature, but in a completely different way than what was done in [8]. First, we will not use Probabilistic Multivariate equations, i.e. each equation of the public key will have to be satisfied. Second, we will have a number of LL' that is different (generally smaller) from the number of equations. We will obtain like this a new scheme with short signatures, called $HFE_{LL'}$.

Then we will introduce an unusual types of public quadratic equations called “Dragon public keys” [10]. Thanks to this, we will be able to avoid birthday paradox attacks in a very efficient way (more efficient than the so called Patarin-Feistel trick used in GeMMs). The scheme obtained, our second new scheme, called *DragonHFE* $_{LL'}$, will now have, as we will see, “ultra-short” signatures. By “ultra short” signatures we mean that the length of the signature is smaller than security parameter multiplied by two. Therefore, in this case, it is necessary to use some kind of trick to avoid birthday attack. Finally, we will quickly discuss the possibility to use public equations of degree 3 instead of 2.

2 HFE

In this section, we will describe Vanilla (or “Nude”) HFE, i.e. HFE without any additional perturbation.

We will use a following notation. As $\mathbb{F}$ we will denote a finite field $\mathbb{F}_q = \mathbb{F}_{p^u}$ for p prime, $u \in \mathbb{N}^*$. The extension field $\mathbb{F}_{q^n}$ of $\mathbb{F} = \mathbb{F}_q$ will be denoted as $\mathbb{E}$. We will also use φ for a canonical isomorphism from $\mathbb{E}$ to the corresponding vector space $\mathbb{F}^n$.

$$\varphi : \mathbb{E} \longrightarrow \mathbb{F}^n$$

such that

$$\varphi : V = \sum_{i=1}^n v_i \vartheta_i \longmapsto (v_1, \dots, v_n),$$

where $(\vartheta_1, \dots, \vartheta_n)$ is a basis of $\mathbb{E}$ as a vector space over $\mathbb{F}$.

In the extension field $\mathbb{E}$, we will define a univariate polynomial H^* over $\mathbb{E}$ as

$$H^* : \mathbb{E} \longrightarrow \mathbb{E}$$

such that

$$H^* : X \longmapsto \sum_{i,j; q^i + q^j \leq D} \alpha_{i,j} X^{q^i + q^j},$$

where $\alpha_{i,j} \in \mathbb{E}$.

Note, that we use a d' to denote the biggest value of i or j . Small d is used to denote rank, which is equal to number of terms X^{2^n+1} . Usually, we pick HFE coefficients with all monomials of form X^{2^i+1} and therefore $d = d'$.

Since H^* has this specific form, we can create a multivariate quadratic polynomial H over $\mathbb{F}$ as $H = \varphi \circ H^* \circ \varphi^{-1}$:

$$H : \mathbb{F}^n \longrightarrow \mathbb{F}^n$$

$$H : (v_1, \dots, v_n) \mapsto (h_1(v_1, \dots, v_n), \dots, h_n(v_1, \dots, v_n)),$$

where $(h_1, \dots, h_n) \in (\mathbb{F}[x_1, \dots, x_n])^n$ are quadratic polynomials

$$H(\varphi^{-1}(x_1, \dots, x_n)) = \varphi^{-1}(h_1, \dots, h_n)$$

$$H\left(\sum_{i=1}^n \vartheta_i x_i\right) = \sum_{i=1}^n \vartheta_i h_i.$$

Note that H^* is selected in such a way that it is easily inverted. Therefore, the maximum degree D must not be too high.

Since we want to hide the structure, we will use two bijective linear maps. Otherwise, it is possible (in polynomial time) to find a solution for $H(x) = a; a \in \mathbb{E}$.

We define two bijective linear maps S and T , which can be represented as two univariate polynomials

$$S(X) = \sum_{i=1}^n s_i X^{q^i},$$

$$T(X) = \sum_{i=1}^n t_i X^{q^i},$$

over $\mathbb{E}$. We will denote as S and T their corresponding matrix representations. Such matrices can be defined for any linear map $A(X)$ from space X_n to space Y_m as a matrix A of a form $m \times n$, where $A(x) = Ax$.

Finally, we define P as a composition of the functions above as

$$P = T \circ H \circ S,$$

more precisely

$$P = T \circ \varphi \circ H^* \circ \varphi^{-1} \circ S,$$

which can be interpreted as a multivariate quadratic system of n equations over n variables

$$\begin{aligned} P_1(a_1, \dots, a_n), \\ P_2(a_1, \dots, a_n), \\ \vdots \\ P_n(a_1, \dots, a_n). \end{aligned}$$

In this settings, we have a public key

$$PK = P = \{P_1, \dots, P_n\}$$

and a secret key

$$SK = \{H^*, S, T\}.$$

Signature and verification algorithm is as follows. Let's assume that the signer wants to sign message M . The Signing procedure goes as follows.

1. Compute hash of the message $B = Hash(t, M)$, where M is a message to be signed and t is a random diversification value.
2. Compute $Y = TB$.
3. We find any solution (X) to the function H^* such that

$$H^*(X) = Y$$

If the solution does not exist, we go to step 1 and try again with different value t .

4. Finally, we invert linear map S and compute $A = S^{-1}(X)$, which is the signature of our message.

Verification algorithm is very simple. The verifier just verifies if

$$\text{Hash}(t, M) = P(A)$$

for message M .

3 $HFE_{LL'}$

In this section, we will describe our first proposal for signatures. We will use HFE for signature with $q = 2$. However, we will not use “Nude” HFE as described in subsection 2, since it is prone to some super-polynomial attacks [5,6]. We will use a typical tool to overcome these problems. We will use perturbations, namely we will use a variant of HFE with LL' perturbations. Our approach is different compared to the approach of Gouget and Patarin in [8] since in our case all the public equations must be satisfied and we don’t use new and independent LL' perturbation per each equation.

Let’s assume that we have

$$P = T \circ H \circ S$$

as described in section 2 for nude HFE. For $1 \leq r \leq n$, let L_i, L'_i be $2r$ secret linear forms for $i \in [1, \dots, r]$. More precisely, we have $L_i(x_1, \dots, x_n)$ and $L'_i(x_1, \dots, x_n)$ as $2r$ secret linear functions of n variables over $\mathbb{F}$. Note that

$$\mathbb{P}[L_i(x_1, \dots, x_n) \cdot L'_i(x_1, \dots, x_n) \neq 0] = \frac{1}{4}$$

for $q = 2$, where $\mathbb{P}[X]$ denotes probability of an event X .

We can now define P'_j as

$$P'_j = P_j + \sum_{i=1}^r a_{i,j} L_i L'_i$$

for $j \in \{1, \dots, n\}$, where the $a_{i,j}$ are secret coefficients. The P'_j polynomials will be the public key.

The private key consists of T, H, S, L, L' and $a_{i,j}$.

Signature is computed in following steps:

1. Compute hash of the message $B = \text{Hash}(t, M)$ with small random diversification value t .
2. Compute $Y = TB$.
3. Select the terms $L_i L'_i$, where $L_i L'_i = 1$. This is called Hypothesis H .
4. Inverse HFE with these selected values of $L_i L'_i$, i.e. find a solution in X of

$$H^*(Y) + A = Y,$$

where $A = T(A')$, where A' is the vector of components $\sum_{i=1}^r a_{i,j} L_i L'_j$. Note that in the case that the solution does not exist, we try again with different value t (in step 1) or different Hypothesis H (in step 3). The effect of this unsuccessful attempts is described in subsection 3.1.

5. Verify if Hypothesis H is fulfilled. It means to test if the selected places of $L_i L'_i$ match with the computed values. If yes, we have a valid signature. Note, that this is selected to a specific probability. If no, we have to start over from the beginning. We will denote by T the average number of attempts before finding a valid signature.
6. Finally, we compute $A = S^{-1}(X)$, which is the signature of our message.

In the first step of the signing process, we are selecting the value for terms $L_i L'_i$. This has to be done with specific probability. This is called Hypothesis H . We must not select all the values $L_i L'_i$ being zero, because there is an attack of degree 2 for that case. More details will be given in the subsection 6.3. So far, we can note that typically we will choose one $L_i L'_i$ being 1 and others being 0.

3.1 Probability of solution

The probability for a given value in $[1, n]$ to be in the image of a random function is

$$1 - \frac{(n-1)^n}{n^n}.$$

For n large this is about

$$1 - \frac{1}{e}$$

(i.e. about 0.632).

For quadratic multivariate functions like us, we have no proof that this is also about the case, however computer simulations have always showed that this is the case.

So, we will have to try about

$$\frac{e}{e-1} \approx 1.582$$

times in average for a given $Hash(t, M)$ to have a solution of the public equations.

In our scheme we can achieve this by trying different small values for t , and signing $Hash(t, M)$.

Another possibility would be to not publish a small number of the equations of what was initially the public key equations (this could also make cryptanalysis more difficult for some attacks). Then the probability that the public equations have a solution for a given $Hash(t, M)$ is quickly near 1.

3.2 Managing value t

During the signature, in the step 1 we compute hash of the message together with value t . This value is usually called salt and it is needed for the verification process as well.

We have a two how to handle this problem. We can either consider this value as a part of a signature. This approach would make this signature a bit longer and since our goal is to make the signature as small as possible, this approach is not following our goal. However, most of the time, it would add less than 3 bits.

The second possibility is not to send the signature. In this case, the verifier would have to try multiple possible values until he would find a correct value for t . This kind of approach is similar to the idea in [11]. However, in this case it would make sense to limit the length of the value t so that the attacker would not be able

to send a random value as a signature and hope that sooner or later the verifier will find some value t that would make a valid signature. The problem with this approach is that if we limit the size of the t , there might exist a message, that will not be able to sign. The probability of that would be extremely low even for relatively small bound for size of t , but it could theoretically happen.

Note that it is important to separate salt t from the message M inside of the hash. If the separation is not clear, the attacker could take valid signature of M_1 and t_1 and claim that it is a signature of message M_2 with salt t_2 , where t_2 are z bits of t_1 (where $z < |t_1|$) and M_2 is a concatenation of $(|t_1| - z)$ right bits of t_1 and M_1 . To mitigate this problem, we can use several approaches. The first option is to use fixed separator (for example 4 consecutive zeros) and value t can be any random value that does not contain this separator. Another possibility is to use any value t , but the value would have to be hashed (which would fix the length) before concatenating with the message M . The process would be done as

$$\text{Hash}(t, M) = \text{Hash}(\text{Hash}(t) \parallel M)$$

where $\parallel$ denotes concatenation.

4 Example of parameters for $HFE_{LL'}$

For 128 bits of security, we suggest following parameters for this variant:

$$q = 2, n = 256, r = 24.$$

With these parameters we have signature of 256 bits. We will see in section 6 why these parameters resist known attacks.

For different levels of security, we propose parameters in table 4.1.

security bits	n	$d + r$	signature size	public key size	d	D	r	T	T'	time [s]
80	160	12	160 b	252 KiB	2	5	10	84	$8 \cdot 10^4$	0.023 s
					3	9	9	63	$1 \cdot 10^5$	0.032 s
100	200	17	200 b	491 KiB	2	5	15	355	$4 \cdot 10^5$	0.124 s
					3	9	14	266	$7 \cdot 10^5$	0.167 s
128	256	24	256 b	1.0 MiB	2	5	22	2661	$4 \cdot 10^6$	1.2 s
					3	9	21	1995	$6 \cdot 10^6$	1.6 s
192	384	40	384 b	3.38 MiB	2	5	38	$3 \cdot 10^5$	$5 \cdot 10^8$	3.0 m
					3	9	37	$2 \cdot 10^5$	$8 \cdot 10^8$	4.0 m
256	512	56	512 b	8.02 MiB	2	5	54	$3 \cdot 10^7$	$7 \cdot 10^{10}$	6.6 h
					3	9	53	$2 \cdot 10^7$	$1 \cdot 10^{11}$	8.9 h

Table 4.1: Parameters for $HFE_{LL'}$ of degree 2, b = bits, B = bytes.

Size of the public key can be computed as

$$|PK| = n \cdot C'(n, 2) = \frac{n^3 + n^2}{2},$$

where $C'(n, k)$ denotes combinations of k items from n elements with repetition.

More details about the computation of value T , which denotes the average number of iterations for obtaining a valid signature will be described in section 8.

Since Berlekamp algorithm has a complexity in

$$\mathcal{O}(D^\omega + D(\log D \log \log D) \cdot n)$$

for $2 < \omega \leq 3$. Since we perform T attempts in average and we have relatively small values D , we will use the signature complexity in

$$\mathcal{O}(T') = \mathcal{O}((D^3 + nD) \cdot T)$$

We can see in table 4.1 that smaller value d^1 (for given $(d + r)$) gives faster computation time. We will give more details about this in subsection 8.1. When we want more than 128 bits of security it seems to be better to have very small value d ($d = 2$ for example)

We also include an approximation of the signature time. It is based on the number of CPU cycles necessary for one attempt to find a solution (C) multiplied by average number by attempts per one valid signature (T). Finally, we use a parameters of Intel Core i7-10850H 2.7GHz CPU to translate the cycles to seconds. Note, that the construction can be easily parallelised, so we expect better results for efficient implementation.

We use a following formula for the estimation of the cycles:

$$time = C \cdot T \cdot \frac{1}{2.7 \cdot 10^9},$$

where C can be estimated based on implementation of GeMSS and it has a values from a table 4.2 for our purposes.

n	D	C
160	5	$8 \cdot 10^5$
160	9	$1 \cdot 10^6$
200	5	$9 \cdot 10^5$
200	9	$2 \cdot 10^6$
256	5	$1 \cdot 10^6$
256	9	$2 \cdot 10^6$
384	5	$2 \cdot 10^6$
384	9	$3 \cdot 10^6$
512	5	$2 \cdot 10^6$
512	9	$4 \cdot 10^6$

Table 4.2: Estimated number of cycles for one attempt of finding a solution of the system for degree 2.

These values were estimated based on number of cycles in [9] and complexity of Berlekamp algorithm. We assumed the case of $D \in \{5, 9\}$, n according to the signature parameters and the number of cycles was adjusted accordingly.

¹ We recall that in our case we have $D = 2^d + 1$.

Since we are using $D \in \{5, 9\}$, it is better for us to use Berlekamp algorithm compared to Cantor/Zassenhaus algorithm. From [12] we can see that complexity of Berlekamp algorithm is

$$\mathcal{O}(D^\omega + D(\log D \log \log D) \cdot n),$$

where $\omega < 3$. We can compare it with the complexity of Cantor/Zassenhaus algorithm, which according to [12] is

$$\mathcal{O}(D^2(\log D \log \log D)(\log D + n)).$$

We can see, that if $D \in \{5, 9\}$, it is better to use Berlekamp algorithm directly from the asymptotic complexities. For $D = 1025$ ($d = 10$) we can see the advantage of Berlekamp algorithm from the comparison tables in [12].

As we can see in table 4.1, signing times are acceptable up to 128 bits of security. For higher values signing is slow. On the other hand, signature verification is very fast and it was not given in table 4.1.

5 Dragon scheme

In this section, we will introduce the second proposal, which consists of combination of LL' perturbations and dragon scheme.

5.1 Dragon scheme

Dragon scheme, as originally shown in [10], is a construction for a public key. Dragon public key is a set of k equations of n variables $a_1, \dots, a_n$ and m variables $b_1, \dots, b_m$ as follows:

$$\begin{aligned} P_1(a_1, \dots, a_n) + Q_1(a_1, \dots, a_n, b_1, \dots, b_m) &= F_1(b_1, \dots, b_m), \\ P_2(a_1, \dots, a_n) + Q_2(a_1, \dots, a_n, b_1, \dots, b_m) &= F_2(b_1, \dots, b_m), \\ &\vdots \\ P_k(a_1, \dots, a_n) + Q_k(a_1, \dots, a_n, b_1, \dots, b_m) &= F_k(b_1, \dots, b_m), \end{aligned} \tag{1}$$

where P_i are polynomials of degree 2 in a_i over $\mathbb{F}$ and Q_i are polynomials, which are of degree 1 in a_i variables and of degree 1 in b_i variables separately while overall degree of polynomial Q_i is 2 in $(a_1, \dots, a_n, b_1, \dots, b_m)$ over $\mathbb{F}$. $F_1, F_2, \dots, F_k$ can be any pseudo-random functions of $b_1, \dots, b_m$ (not necessarily polynomials of small degree) and they can be generated from a small seed (so they can increase the length of the public key with only the length of the seed, 128 bits for example)

What is important to notice is that functions $F_1, F_2, \dots, F_k$ do not depend on the secret and therefore cannot lead to any attack and that they can be generated from a small seed.

Another variant can be to use the polynomials $P_1, \dots, P_k, Q_1, \dots, Q_k$ as the seed, so it does not increase the size of the public key at all. (Moreover, in our schemes it seems to be possible to choose for example the simple function $F_i = b_i$.)

It is possible to obtain such public equations (1) from secret equations (2) like this:

$$\sum_{0 \leq i,j \leq d} \alpha_{i,j} X^{q^i + q^j} + \sum_{i=0}^d \sum_{j=1}^k \beta_{i,j} X^{q^i} Y_j = 0, \quad (2)$$

where Y_j is secret random linear expression in $y_1, \dots, y_m$, but we will below introduce LL' perturbations.

Consequently, we get a set of equations (1). Note that here $k \approx n$ and $m \approx 2n$.

To verify a signature $(a_1, \dots, a_n)$ of a message M , a verifier simply checks if all the equations (1) are satisfied while $(b_1, \dots, b_m) = \text{Hash}(M)$.

However, to make a real public key, we have to hide this structure similarly as we did in 2. We will mask it by two secret linear bijective maps which are represented by two secret invertible matrices S and T . We set

$$\begin{aligned} X &= SA, \\ Y &= TB, \\ F &= U(F'), \end{aligned}$$

where

$$A = (a_1, \dots, a_n)$$

will be a signature of a message M and

$$B = (b_1, \dots, b_m) = \text{Hash}(M)$$

hash of the message M .

These linear maps will make the public key a bit more robust. Nevertheless, this is not enough, since similarly to nude HFE we will have here the analogy of MinRank and Gröbner super-polynomial attacks. So we will describe one additional technique in the following subsection.

5.2 Dragon $HFE_{LL'}$ scheme

We will start with the same way as in the previous subsection. We have a standard HFE public key set of equations

$$\begin{aligned} P_1(x_1, \dots, x_n) + Q_1(x_1, \dots, x_n, y_1, \dots, y_m), \\ \vdots \\ P_k(x_1, \dots, x_n) + Q_k(x_1, \dots, x_n, y_1, \dots, y_m). \end{aligned} \quad (3)$$

We construct secret linear forms in $(a_1, \dots, a_n, b_1, \dots, b_m)$ L_i and L'_i for $i \in \{1, \dots, r\}; 1 \leq r \leq n$.

Note that $\mathbb{P}(L_i L'_i \neq 0) = \frac{1}{4}$ for $\mathbb{F} = GF(2)$ again. For this reason, we will focus on this case $\mathbb{F} = GF(2)$.

We now construct a secret linear combinations of $L_i L'_i$ that we call V_i for $i \in \{1, \dots, k\}$.

We define polynomials R_i as

$$R_i = \sum_{j \in \{1, \dots, m\}} \epsilon_{i,j} b_j, \quad (4)$$

where

$$V_i = \sum_{j=1}^n \sum_{c \in \{a_1, \dots, a_n, b_1, \dots, b_m\}} \delta_{j,c} a_j c + \sum_{l,j \in \{1, \dots, m\}} \epsilon_{l,j} b_l b_j. \quad (5)$$

In other words, we are setting R_i as a part of the V_i which does not contain any a_j value.

We can define V_i^* as

$$V_i^* = V_i - R_i \quad (6)$$

If we denote V, V^* and R as

$$\begin{aligned} V &= (V_1, \dots, V_k) \\ V^* &= (V_1^*, \dots, V_k^*) \\ R &= (R_1, \dots, R_k) \end{aligned} \quad (7)$$

we can write a public key as

$$\begin{aligned} P_1(a_1, \dots, a_n) + Q_1(a_1, \dots, a_n, b_1, \dots, b_m) + V_1^*(a_1, \dots, a_n, b_1, \dots, b_m) &= \\ &= F_1(b_1, \dots, b_m), \\ &\vdots \\ P_k(a_1, \dots, a_n) + Q_k(a_1, \dots, a_n, b_1, \dots, b_m) + V_k^*(a_1, \dots, a_n, b_1, \dots, b_m) &= \\ &= F_k(b_1, \dots, b_m). \end{aligned} \quad (8)$$

Secret key consists of $\{V, R, S, T, H\}$.

Signing mechanism is as follows:

1. We compute $B = \text{Hash}(t, M)$, where M is a message to sign.
2. We compute the values Y_i and R from B .
3. We choose which $L_i L'_i$ are at 0 (with probability $\frac{3}{4}$) and which $L_i L'_i$ are at 1 (with probability $\frac{1}{4}$). We call this Hypothesis H . (Often, we will choose only one $L_i L'_i$ at 1 and others at 0. More details can be found in [6.3](#))
From this hypothesis H , we can compute the vector V (since the components of V are linear combination of $L_i L'_i$ values).
4. We now solve the equation

$$\sum_{0 \leq i, j \leq d} \alpha_{i,j} X^{2^i + 2^j} + \sum_{i=0}^d \sum_{j=1}^m \beta_{i,j} X^{2^i} Y_j + V + R = F \quad (9)$$

in X . (If there is no solution, go to step [3](#) and choose different Hypothesis H or start over with different value t .)

5. We compute $A = S^{-1}X$.
6. We check if Hypothesis H is satisfied (probability of this is $\frac{3^{r-1}}{4^r}$). If yes, A is a valid signature of M . If no, we try this with another Hypothesis H (in step [3](#)) or we start over with different value of t in the step [1](#).

Note that in step [5](#) we can have more than one solution X . In that case, we can pseudo-randomly choose one of them. Moreover, t , Hypothesis H and this X can be fixed for M , i.e. the same message M will always have the same signature.

Note that thanks to R_i all $b_i b_j$ terms were removed except the terms that may come from F_i .

5.3 Small examples

In this subsection, we will show three very small examples to better illustrate the principle.

Example 1:

$$a_9X^9 + a_6X^6 + a_5X^5 + a_3X^3 + b_4X^8Y_4 + b_3X^4Y_3 + b_2X^2Y_2 + b_1XY_1 = 0$$

Where Y_1, Y_2, Y_3, Y_4 are four vectors of which components are secret linear terms in $y_1, y_2, \dots, y_m$ and where the a_i and b_j are values in the extension field $\mathbb{E} = GF(2^n)$.

Here, $D = 9$, and it will give quadratic Dragon equations since $9 = 8 + 1, 6 = 4 + 2, 5 = 4 + 1$ and $3 = 2 + 1$ are sums of two powers of 2 and since 8, 4, 2, 1 are powers of 2 directly. We can see that d in this case is equal to $d = 3$, because we have three terms of the form X^{2^i+1} (i.e. X^3, X^5, X^9). Note that $X^6 = (X^3)^2$ is a linear transform of X^3 and therefore X^6 does not increase the rank d .

Example 2:

$$a_3X^3 + b_2X^2Y_2 + b_1XY_1 = 0$$

This is a minimalist example with $D = 3$ and $d = 1$. Since it is very fast to compute roots of this equations of degree 3 and as we will see, we want to have d as small as possible, at present we can consider this to be the best option.

6 Cryptanalysis

In this section, we will analyze different attacks on the proposed signature scheme. First of all, we have to consider different kinds of MinRank attacks, we should make a special case about MinRank attack against xy term. After that, we will analyze direct attack based on Gröbner basis. We will also describe the applicability of a Birthday attack and finally, we will have a look on the polarization attack of the degree 3.

6.1 MinRank Attacks

MinRank attack is based on the fact that the signer's secret is a low-degree polynomial over an extension field, obfuscated by affine transformations to produce a public system of quadratic equations. This algebraic structure offers a possibility to offers the attacker to search for the structure. In MinRank attacks, the attacker is looking for the low-rank linear combinations of the public quadratic forms in order to remove the S and T affine maps. The attack on S layer was first described in [6] and the attack against T was described in [5].

Here we would like to study the practical complexity of these methods against our proposals. The way of solving the MinRank problem using the support minor method is described in [13] and [14]. Since we are using the same type of LL' perturbations as [9], we conclude that we can use the same parameter r (number of perturbations) for given security levels when taking into account the parameter d . This is shown in table 6.1. We can see in the complexity formula

$$\mathcal{O} \left((d+r)(n-1)^4 \binom{2(d+r)+1}{d+r}^2 \right)$$

that the important term is $r + d$ and we can have a trade-off between these two parameters. More details about this trade-off is described in 8.1.

security level	$d + r$
80	12
100	17
128	24
192	40
256	56

Table 6.1: Number of perturbations (parameter r) necessary for given security levels for given d .

MinRank attack against xy term We assume that the LL' perturbations should make the attack against xy term fail. At the same time, the assumption should be proven and that is the goal of this paragraph.

The $x_i y_j$ terms (that comes from the “Dragon terms”) are quite new in multivariate cryptography, and therefore it is not easy to evaluate if they can lead to devastating attacks or not (since no paper so far evaluated MinRank attacks or other attacks on these terms).

However, our feeling at present is that MinRank attacks on these terms will probably have about the same complexity as for $x_i x_j$ terms (where we have seen that the complexity is related to $r + d$).

This comes from the fact that the rank on $x_i x_j$ attacks comes from the number d of monomials of the form $X^{(1+2^i)}$, and we have d terms like this up to degree $D = 1 + 2^d$.

Similarly, we expect that the rank on $x_i y_j$ attacks comes from the number of monomials of the form $X^{2^i} Y_i$ where Y_i components are random linear terms in the y_j variables, and we have here again d terms like this up to degree $D = 1 + 2^d$.

If in the future a devastating attack on these “Dragon terms $x_i y_j$ ” is found, then maybe we will have to give up this Dragon form and to go back to the first scheme of this paper (with larger signature but no dragon terms).

6.2 Direct Attack

Direct algebraic attacks on HFE signatures frame signature forgery as solving the public multivariate quadratic system for the preimage of a target value. To do so, the attacker uses Gröbner-basis algorithms. By modeling the public equations as a polynomial ideal, attackers apply different techniques as described in [15] and [16] to compute a Gröbner basis and consequently recover the hidden solution.

Again, since we are using the same type of perturbation as in [9], we assume that we can use the same results and finally, we agree with the conclusion that the LL' perturbations works as an efficient obstacle for the attacker.

6.3 LL' recovery

At this subsection, we will focus on the Hypothesis H , which is used during the signature generation in 3 and 5.2.

Let's assume that all $L_i L'_i$ are equal to 0. Then, an attacker can find these LL' perturbations by looking for equations of total degree 2 in a_i and b_j that are always 0.

The complexity (with usage of Gaussian reduction) is about $\mathcal{O}(n^6)$. Note, that some asymptotically, some improvements are possible.

This is the reason, why we have to include some LL' perturbation at 1. This can be done for example with probability 0.01 - 0.05, which seems sufficient.

However, if we always include only one $L_i L'_i$ being 1, then an attacker can compute $LL' L'' L'''$ that are always 0. This can be done by multiplying two pairs, which gives us a complexity $\mathcal{O}(n^4)$.

Similarly, if always at most 2 LL' are 1, the attacker can look for the equations of degree 6 etc.

This is why we decide in some scheme to put one 1 with probability about $\frac{1}{20}$, two 1 with probability about $(\frac{1}{20})^2$, three 1 with probability about $(\frac{1}{20})^3$ etc.

This way, we expect to avoid all attacks with a complexity smaller than the required level of security and to have an average complexity to sign still about the same as when only 1 value LL' is at 1.

Note, that more precise analysis should be done in future.

6.4 Random System Attack

In this part, we would like to discuss the attack that would work even for completely random system. This kind of attack is based on classical Gröbner basis as was pointed out in [11].

In this setting, we assume, that we are working with a “perfect trapdoor”, and the attacker wants to forge a signature by solving a polynomial system of equations.

In our case, we have a public equations, that contain secret trapdoor function (the hidden structure of the public key). This secret structure may offer some advantage to the attacker. For this part, we will assume that the hidden structure does not provide any advantage and therefore the attack will work the same way against the completely random system. This way, we will have a minimal bound for the signature length.

We assume, that the system of equations is completely random, therefore the attacker can invert it by using only generic polynomial solver as for example Gröbner basis algorithm. It means that the complexity of the attack is given by the complexity of the algorithm for solving the polynomial system of equations.

We can solve the system by using Gröbner basis algorithm as is described in [17,18,19]. The complexity of the computation depends on the degree of regularity d_{reg} . The overall complexity is

$$\mathcal{O}\left(\binom{n + d_{reg}}{d_{reg}}^\omega\right),$$

where $2 \leq \omega \leq 3$ is a constant as pointed out in [20] and [21].

At the same time, we can use so-called hybrid approach. We can combine the Gröbner basis attack with exhaustive search to achieve better performance as used in [22]. The complexity decreases and as shown in [11] we can get the values for the shortest possible signature sizes as shown in the second line of a table 6.2, which refers to the degree 2. The third line of the table 6.2 is specific for degree 3 and it was computed in a similar way.

Based on this table, we will propose some explicit parameters in section 8.

		bits of security				
		80	100	128	192	256
degree 2		86	112	145	218	290
degree 3		82	103	131	196	262

Table 6.2: Table of minimum signature length for $q = 2$ and $\omega = 2.37$.

6.5 Birthday paradox

Since with our Dragon $HFE_{LL'}$ scheme we have “ultra-short” signatures ,i.e. a signature length is smaller than 2 times the security level, we must be very careful about the birthday paradox attack.

There are different types of birthday attacks and we will now explain how we chose parameters for our constructions to avoid these attacks.

First birthday attack Let assume that we have (for example) 128 equations on $GF(2)$ of the form:

$$Q_1(x_1, \dots, x_{128}) = y_1$$

...

$$Q_{128}(x_1, \dots, x_{128}) = y_{128}$$

Step 1: generate 2^{64} values X , and compute the corresponding $Q(x_1 \dots x_{128}) = Q'$

Step 2: generate 2^{64} messages M , and compute the corresponding hash values Y

Step 3: look for a collision on the form $Q' = Y$

When this collision is found, you have a signature X of one of the message M . Complexity here: about 2^{64} computations and 2^{64} memory. Why this attack fails on Dragons: because on Dragon public equations we do not have the possibility to have only terms in x on the left side of the equalities, and only terms in y on the other side. We have mixed terms in xy .

Second birthday attack Let assume that we have Dragon public keys on $GF(2)$ with 128 equations, 128 variables x , and 256 variables y .

We look for a collision on the hash values.

Here it does not work because even though we have only 128 equations, $Y = Hash(M)$ has 256 bits and therefore may resist collision attacks up to 128 bits of security.

So here the size of the signature X has only 128 bits but the Hash of the messages have 256 bits to resist collision attacks.

Third birthday attack There is a technique used in [11], which uses signatures, which are even shorter than the security parameter. It is based on the idea that the signature is just a part of a value that can be used for verification. It means that during the verification, it is necessary to find the missing part in order to verify successfully. This makes the verification process ineffective, but at the same

time it adds a complexity to the attacker, who wants to forge a signature. If the attacker wants to be sure that the forged signature is correct, it is necessary to go through the verification step as well and this additional verification complexity compensates for the gap between the signature length and security level.

We don't use this technique here since there might be a possible attack against this setting.

Let's say (just as an example) that we have a security level of 2^{100} and that the signature size is only 80 bits. It means that if the attacker sends a lot of message messages with completely random signatures there is a probability that one of them will be valid even if the attacker does not try to validate it. This kind of attack is not very practical since in this example the number of messages to be sent would have to be huge.

At the same time, this technique can't be taken to the extreme. If we try the extreme case of signature of length 1 bit, we can clearly see that if the attacker sends any message with random signature (this one bit), he will have probability 2^{-1} that the signature is valid. Therefore in order to succeed he just needs to send the message twice with different bit and one of them would seem valid. This means that one must be careful with this technique and consider the trade-off between the computation complexity, space complexity and bandwidth complexity.

6.6 Need for hash

If we fix X we have just linear equations in Y , so if Y was not a hash but just the message then it would be very easy to find a message with a valid signature. However, since the Hash function is supposed to be a one-way function, this do not work.

6.7 Remark

We may also consider Dragon public equations with terms quadratic in y .

So far, we did not see why to add such terms (it will also increase the size of the public key by about 25% if we keep the same number of y variables) but maybe in the future we will like such variants. For example, it may be possible here to have no hash function and then Y will be directly the message. It may also be possible to have Y with only 128 bits instead of 256 bits. This is to be studied, but this is not our main scheme at present.

7 Equations of higher degree

We may also consider the public equations of degree 3 instead of 2 (as they were described so far). In this case, we can use perturbations of form $L_i L'_i L''_i$, where L_i , L'_i and L''_i are secret linear forms in x_i ; $i \in \{1, \dots, n\}$.

As a consequence, the public key will grow considerably. At the same time, probability of the $LL'L''$ perturbation being 0 grows as well. Specifically from $\frac{3}{4}$ to $\frac{7}{8}$. It means that the size of the signature decreases, which is our main objective, but at a cost of a much longer public key.

7.1 Cryptanalysis

Usually, in multivariate cryptography with public keys of degree 3, we use a differential cryptanalysis (i.e. polarization attack) in order to obtain equations of degree 2, and then to use the usual cryptanalysis on degree 2. Let see what we obtain here like this.

We approach it the same way as it was done in [9] and ignore terms of degree 1 for simplicity.

Let's have $X = (x_1, \dots, x_n)$, $A = (a_1, \dots, a_n)$ and let $f(x) = x_1x_2x_3$ as a simplified notation of $L_iL'_iL''_i$. The differential $f(X + A) + f(A)$ is:

$$(x_1 + a_1)(x_2 + a_2)(x_3 + a_3) + x_1x_2x_3 = a_1x_2x_3 + x_1a_2x_3 + x_1x_2a_3$$

If all $a_i = 0$; $i \in 1, 2, 3$, the rank is 0. If only one a_i is 0, the rank is 1. If there are exactly 2 a_i equal to 0, the rank is still 1, because $x_1x_3 + x_2x_3 = x_3(x_1 + x_2)$. The rank increases to 2 for $a_i = 1$; $i \in 1, 2, 3$. It is not 3, because $x_1x_2 + x_1x_3 + x_2x_3 = x_1x_2 + x_3(x_1 + x_2)$. From there, we can compute the average rank as

$$0 \cdot \frac{1}{8} + 1 \cdot \frac{3}{8} + 1 \cdot \frac{3}{8} + 2 \cdot \frac{1}{8} = 1.$$

It means that we get the same number of terms $\hat{L}\hat{L}'$ in differential as the original terms $LL'L''$. For that reason, we should keep the same value of r for the same level of security.

We may also consider equations of degree 4, 5, 6 and so on, but we find those unnecessary, since we already reached almost the smallest signature size with degree 3 and increasing the degree would only increase the size of public key.

8 Proposed explicit parameters for Dragon $HFE_{LL'}$

In this section, we define parameters for the proposed signatures based on the results of section 6.

The summary of the proposed parameters can be found in tables 8.1 and 8.2

security bits	n	m	$d + r$	signature size	public key size	d	D	r	T	T'	time
80	100	200	12	100 b	306 KiB	2	5	10	84	$5 \cdot 10^4$	0.015 s
						3	9	9	63	$1 \cdot 10^5$	0.02 s
100	128	256	17	128 b	641 KiB	2	5	15	355	$3 \cdot 10^5$	0.079 s
						3	9	14	266	$5 \cdot 10^5$	0.107 s
128	160	320	24	160 b	1.22 MiB	2	5	22	2661	$2 \cdot 10^6$	0.742 s
						3	9	21	1995	$4 \cdot 10^6$	1.0 s
192	230	460	40	230 b	3.63 MiB	2	5	38	$3 \cdot 10^5$	$3 \cdot 10^8$	1.8 m
						3	9	37	$2 \cdot 10^5$	$6 \cdot 10^8$	2.4 m
256	300	600	56	300 b	8.05 MiB	2	5	54	$3 \cdot 10^7$	$4 \cdot 10^{10}$	3.8 h
						3	9	53	$2 \cdot 10^7$	$7 \cdot 10^{10}$	5.2 h

Table 8.1: Parameters for $D - HFE_{LL'}$ of degree 2, b = bits, B = bytes.

security bits	n	m	$d + r$	signature size	public key size	d	D	r	T	T'	time
80	82	164	12	82 b	17.2 MiB	2	7	10	42	$4 \cdot 10^4$	0.008 s
						3	11	9	37	$8 \cdot 10^4$	0.012 s
						4	19	8	32	$3 \cdot 10^5$	0.017 s
100	103	206	17	103 b	42.8 MiB	2	7	15	82	$9 \cdot 10^4$	0.021 s
						3	11	14	72	$2 \cdot 10^5$	0.028 s
						4	19	13	63	$6 \cdot 10^5$	0.043 s
128	131	262	24	131 b	112 MiB	2	7	22	209	$3 \cdot 10^5$	0.067 s
						3	11	21	183	$5 \cdot 10^5$	0.092 s
						4	19	20	160	$1 \cdot 10^6$	0.139 s
192	196	392	40	196 b	559 MiB	2	7	38	1770	$3 \cdot 10^6$	0.847 s
						3	11	37	1549	$5 \cdot 10^6$	1.2 s
						4	19	36	1355	$1 \cdot 10^7$	1.8 s
256	262	524	56	262 b	1.74 GiB	2	7	54	$1 \cdot 10^4$	$3 \cdot 10^7$	9.6 s
						3	11	53	$1 \cdot 10^4$	$6 \cdot 10^7$	13.2 s
						4	19	52	$1 \cdot 10^4$	$1 \cdot 10^8$	19.9 s

Table 8.2: Parameters for $D - HFE_{LL'}$ of degree 3, b = bits, B = bytes. Value d in this table is the rank of the differential, which is of degree 2.

The length of the public key for degree 2 can be computed as

$$|PK| = n \cdot (C'(n, 2) + n \cdot m) \stackrel{(m=2n)}{=} \frac{5}{2}n^3 + \frac{1}{2}n^2,$$

where $C'(n, k)$ denotes combination of k items from n elements with repetition.

If we consider the public key of a form

$$\sum_{0 \leq i, j \leq d} \alpha_{i,j} X^{q^i + q^j} + \sum_{i=0}^d \sum_{j=1}^k \beta_{i,j} X^{q^i} Y_j = 0$$

we can see that the value $C'(n, 2)$ is the number of terms $\alpha_{i,j}$. The value $(n \cdot m)$ is the number of terms $\beta_{i,j}$. In total, we have n rows like this.

For degree 3 it is

$$|PK| = n \cdot (C'(n, 3) + m \cdot C'(n, 2) + n \cdot C'(m, 2)) \stackrel{(m=2n)}{=} \frac{19}{6}n^4 + \frac{5}{2}n^3 + \frac{1}{3}n^2.$$

We can see this from the public key

$$\sum_{0 \leq i, j, l \leq d} \alpha_{i,j,l} X^{q^i + q^j + q^l} + \sum_{0 \leq i, j \leq d} \sum_{l=1}^k \beta_{i,j} X^{q^i + q^j} Y_l + \sum_{0 \leq i \leq d} \sum_{0 \leq j, l \leq k} \gamma_{i,j,l} X^{q^i} Y_{j,k} = 0$$

we can see that the value $C'(n, 3)$ is the number of terms $\alpha_{i,j,l}$. The value $(m \cdot C'(n, 2))$ is the number of terms $\beta_{i,j,l}$ and the value $(n \cdot C'(m, 2))$ is the number of terms $\gamma_{i,j,l}$. In total, we have n rows like this.

Note, that we eliminated last quadratic term by design in degree 2 and we did the same for the last term of degree 3.

Value T denotes the average number of iterations for obtaining a valid signature.

Let's first focus on the case of degree 2. In order to estimate this value, we will assume that we have exactly one index j such that $L_j L'_j$ value is equal to 1 and the others $L_i L'_i, i \neq j$ are equal to 0. If we get the values of L_i and L'_i randomly, the probability for the product to be 1 is $\frac{1}{4}$.

Let's first focus on the value of $L_j L'_j$. The probability of this value being correct is $\frac{1}{4}$. For all the other indices, the probability of the product $L_i L'_i$ being correct is $\frac{3}{4}$.

Therefore, the overall probability can be estimated as $\frac{1}{4} \cdot \left(\frac{3}{4}\right)^{r-1}$ since we have r perturbations $L_i L'_i$.

Now, we can do the same estimation for degree 3. In this case, the probability of $L_i L'_i L''_i$ being 1 is $\frac{1}{8}$. With this modification, we can update the formula for the probability estimation. The overall probability for degree 3 can be estimated as $\frac{1}{8} \cdot \left(\frac{7}{8}\right)^{r-1}$.

Finally, we will also consider the probability from subsection 3.1. It means that for degree 2 we will use

$$T = \frac{e}{e-1} \cdot \frac{4^r}{3^{r-1}}$$

and for degree 3, we have

$$T = \frac{e}{e-1} \cdot \frac{8^r}{7^{r-1}}$$

in the tables, which takes into an account the probability of having a solution for given value of t .

It means that the value T can be estimated as $1.582 \cdot \frac{4^r}{3^{r-1}}$ for degree 2 and as $1.582 \cdot \frac{8^r}{7^{r-1}}$ for degree 3.

Note, that the real value of T is slightly different since we assumed that we have exactly one LL' equal to 1.

We also use value T' as described in section 4, which should better estimate the computation cost.

n	D	C
100	5	$5 \cdot 10^5$
100	9	$8 \cdot 10^5$
128	5	$6 \cdot 10^5$
128	9	$1 \cdot 10^6$
160	5	$8 \cdot 10^5$
160	9	$1 \cdot 10^6$
230	5	$1 \cdot 10^6$
230	9	$2 \cdot 10^6$
300	5	$1 \cdot 10^6$
300	9	$3 \cdot 10^6$

Table 8.3: Estimated number of cycles for one attempt of finding a solution of the system for degree 2.

n	D	C
82	7	$5 \cdot 10^5$
82	11	$8 \cdot 10^5$
82	19	$1 \cdot 10^6$
103	7	$7 \cdot 10^5$
103	11	$1 \cdot 10^6$
103	19	$2 \cdot 10^6$
131	7	$9 \cdot 10^5$
131	11	$1 \cdot 10^6$
131	19	$2 \cdot 10^6$
196	7	$1 \cdot 10^6$
196	11	$2 \cdot 10^6$
196	19	$4 \cdot 10^6$
262	7	$2 \cdot 10^6$
262	11	$3 \cdot 10^6$
262	19	$5 \cdot 10^6$

Table 8.4: Estimated number of cycles for one attempt of finding a solution of the system for degree 3.

Finally, we estimate the time needed for signature generation. We used the same principle as in section 4. Exact values of C used in our computation can be found in tables 8.3 and 8.4.

As we can see in table 8.2 in degree 3, public key sizes are large, but the times of signature generations are acceptable even for bigger security parameters.

8.1 Trade-off of parameters d and r

At this subsection, we will discuss the influence of two parameters. Namely, we will describe the effect of parameter d and parameter r .

Parameter d depends on D , which is the total degree of the polynomial in the extension field $\mathbb{E}$. If we have a degree 2 equations in the small field $\mathbb{F}$ it means that each power p in the extension field $\mathbb{E}$ can be computed as $p = q^i + q^j$, where $i, j \leq d$. This means that each power p in $\mathbb{E}$ is limited by $D < 2 \cdot 2^d$. Similarly for degree 3. Each power p in $\mathbb{E}$ can be computed as $p = q^i + q^j + q^k$, where $i, j, k \leq d$ and therefore $D < 3 \cdot 2^d$.

Parameter r describes the number of perturbations $L_i L'_i$ that we use in the system.

They both have an influence on the performance and security.

From the security point of view, we saw in section 6 that we need to reach at least given bound for $d + r$ for given security parameter (more details are given in table 6.1). It means that we can have a trade-off between d and r for given security level.

From the performance point of view, the bigger value of r , the higher number of attempts to get valid signature. On the other hand, the smaller value r , the bigger

value d . With increasing d the complexity of solution finding increases as well. We can estimate it by the Berlekamp algorithm as we did above.

From the tables, we can clearly see, that for fixed security level the optimal selection is to have d as small as possible (that is $d = 2$).

9 Conclusion

We have proposed two new variants of signature. The first one is $HFE_{LL'}$ and the second one is $D - HFE_{LL'}$. Both variants are trying to minimize the size of the signature using LL' perturbations. The second variant achieves “ultra short” signatures by applying “Dragon scheme”. This way we got for example signature sizes of 256, 160 and 131 bits for 128 bits of security.

References

1. Beullens, W., Chen, M.S., Ding, J., Gong, B., Kannwischer, M.J., Patarin, J., Peng, B.Y., Schmidt, D., Shih, C.J., Tao, C., Yang, B.Y.: UOV: Unbalanced oil and vinegar - algorithm specifications and supporting documentation, version 2.0
2. Beullens, W., Campos, F., Celi, S., Hess, B., Kannwischer, M.J.: MAYO - round 2 version
3. Wang, L.C., Chou, C.Y., Ding, J., Kuan, Y.L., Leegwater, J.A., Li, M.S., Tseng, B.S., Tseng, P.E., Wang, C.C.: SNOVA - proposal for NISTPQC: Additional digital signature schemes. version 2
4. Casanova, A., Faugère, J.C., Macario-Rat, G., Patarin, J., Perret, L., Ryckeghem, J.: GeMSS: A great multivariate short signature (2025) Accessed 09-01-2026.
5. Bettale, L., Faugère, J.C., Perret, L.: Cryptanalysis of multivariate and odd-characteristic hfe variants. In Catalano, D., Fazio, N., Gennaro, R., Nicolosi, A., eds.: Public Key Cryptography – PKC 2011, Berlin, Heidelberg, Springer Berlin Heidelberg (2011) 441–458
6. Tao, C., Petzoldt, A., Ding, J.: Efficient key recovery for all hfe signature variants. In Malkin, T., Peikert, C., eds.: Advances in Cryptology – CRYPTO 2021, Cham, Springer International Publishing (2021) 70–93
7. Cogliati, B., Macarior-Rat, G., Patarin, J., Varjabedian, P.: State of the art of HFE variants is it possible to repair HFE with appropriate perturbations? Cryptology ePrint Archive, Paper 2024/1706 (2024)
8. Gouget, A., Patarin, J.: Probabilistic multivariate cryptography. Volume 4341. (01 2006) 1–18
9. Patarin, J., Varjabedian, P.: Multivariate encryptions with LL' perturbations - is it possible to repair HFE in encryption? -. Cryptology ePrint Archive, Paper 2024/1999 (2024)
10. Patarin, J.: Asymmetric cryptography with a hidden monomial. In: Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings. Volume 1109 of Lecture Notes in Computer Science., Springer (1996) 45–60
11. Patarin, J., Macario-Rat, G., Bros, M., Koussa, E.: Ultra-short multivariate public key signatures. Cryptology ePrint Archive, Paper 2020/914 (2020)

12. Shoup, V.: Factoring polynomials over finite fields: asymptotic complexity vs. reality. In: Proceedings of the IMACS Symposium. (1993) 124–129
13. Baena, J., Briaud, P., Cabarcas, D., Perlner, R., Smith-Tone, D., Verbel, J.: Improving support-minors rank attacks: applications to geMSS and rainbow. Cryptology ePrint Archive, Paper 2021/1677 (2021)
14. Bardet, M., Briaud, P., Bros, M., Gaborit, P., Tillich, J.P.: Revisiting algebraic attacks on MinRank and on the rank decoding problem. Cryptology ePrint Archive, Paper 2022/1031 (2022)
15. Faugere, J.C.: A new efficient algorithm for computing gröbner bases (f4). Journal of pure and applied algebra **139**(1-3) (1999) 61–88
16. Faugere, J.C.: A new efficient algorithm for computing gröbner bases without reduction to zero (f 5). In: Proceedings of the 2002 international symposium on Symbolic and algebraic computation. (2002) 75–83
17. Sturmfels, B.: What is... a grobner basis? Notices-American Mathematical Society **52**(10) (2005) 1199
18. Bardet, M., Faugère, J.C., Salvy, B., Spaenlehauer, P.J.: On the complexity of solving quadratic boolean systems. Journal of Complexity **29**(1) (2013) 53–75
19. Bardet, M., Faugere, J.C., Salvy, B.: On the complexity of gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proceedings of the International Conference on Polynomial System Solving. (2004) 71–74
20. Bardet, M.: Étude des systèmes algébriques surdéterminés. Applications aux codes correcteurs et à la cryptographie. PhD thesis, Université Pierre et Marie Curie-Paris VI (2004)
21. Faugere, J.C.: A new efficient algorithm for computing gröbner bases without reduction to zero (f 5). In: Proceedings of the 2002 international symposium on Symbolic and algebraic computation. (2002) 75–83
22. Bettale, L., Faugere, J.C., Perret, L.: Hybrid approach for solving multivariate systems over finite fields. Journal of Mathematical Cryptology **3**(3) (2009) 177